

Environment Skills Catalog

Complete reference of all skills available in this Grok sandbox environment
Generated on September 24, 2026

Overview

This document lists every skill installed in the current environment. Skills are modular instruction packages that specialize the agent for specific tasks. There are two categories:

- **Bundled skills** — shipped defaults (docx, ffmpeg, pdf, pptx, skill-creator, xlsx). Editable in-session but changes do not persist across sessions.
- **User custom skills** — located in `/home/workdir/.grok/skills/`; persist across sessions and can override bundled skills of the same name.

Quick Reference

Skill	Type	Primary Purpose
docx	Bundled	Create, edit, analyze Word documents (.docx / .dotx)
ffmpeg	Bundled	Media processing: convert, trim, resize, merge, GIF, audio
pdf	Bundled	Read, create, transform PDFs (text, tables, forms, OCR)
pptx	Bundled	Create, edit, combine PowerPoint presentations
skill-creator	Bundled	Create and validate new skills
xlsx	Bundled	Create, edit, analyze spreadsheets (.xlsx, .csv, etc.)
coding-agents-harness	User	Launch & manage AI coding agent CLIs
live-preview	User	Serve HTML/CSS/JS or React for live browser preview
nvidia-llm	User	Call NVIDIA Integrate API (minimax / stepfun models)

Bundled Skills

1. docx

Location: `/root/.grok/skills/docx/`

Description: Use whenever the user wants to create, read, edit, or manipulate Word documents (.docx or .dotx files). Triggers include mentions of 'doc', 'Word doc', '.docx', '.dotx', 'Word template', or requests for professional documents with formatting (tables of contents, headings, page numbers, letterheads). Also covers extracting/reorganizing content, inserting/replacing images, find-and-replace, tracked changes/comments, and converting content into polished Word documents. Do **not** use for PDFs, spreadsheets, Google Docs, or general coding.

Key capabilities:

- Create new documents from scratch with docx-js (Node)
- Edit existing templates by unpacking XML, replacing text/fields, packing
- Convert .doc → .docx, accept tracked changes, extract text via pandoc
- Tables, images (with aspect-ratio preservation), headers/footers, TOC, hyperlinks, footnotes

- Validation and visual QA via LibreOffice + pdftoppm

2. ffmpeg

Location: /root/.grok/skills/ffmpeg/

Description: Media processing with ffmpeg/ffprobe — inspect, convert, trim, resize, compress, extract frames/audio, replace audio, mute, make GIFs, add subtitles/overlays, and combine videos. Triggers on phrases such as 'combine these videos', 'merge my clips', 'compress video', 'extract audio', 'make gif', 'resize video', 'thumbnail', 'storyboard', codec settings, etc.

Key capabilities:

- Safe defaults: -n (no overwrite), temp-file workflow, path quoting, ffprobe verification
- Concatenate videos (same-codec copy or mixed-codec re-encode)
- Trim (keyframe-aligned or frame-accurate), fade in/out, resize, extract frames/thumbnails
- Web-compatible MP4 defaults (H.264 + AAC + yuv420p + faststart)
- Decision tree for copy vs re-encode; extensive failure-mode fixes

3. pdf

Location: /root/.grok/skills/pdf/

Description: Read, create, and transform PDF files. Covers pulling text and tables, generating new PDFs, merging/splitting, rotating pages, watermarking, encrypting/removing passwords, extracting embedded images, OCR on scanned documents, and filling out PDF forms (including official tax forms). Apply whenever a task involves a .pdf file.

Key capabilities:

- Extract text (pdfplumber / pdftotext) and tables (→ pandas)
- Combine, split, rotate, watermark, password-protect (pypdf / qpdf)
- Generate new PDFs with reportlab (Canvas or Platypus flowables)
- OCR scanned PDFs (pytesseract + pdf2image)
- Form filling (see forms.md) and visual verification via pdftoppm

4. pptx

Location: /root/.grok/skills/pptx/

Description: Use any time a .pptx file is involved as input or output — create, read, edit, combine, or split presentations, decks, and slides. Triggers on 'deck', 'slides', 'presentation', 'PPT', 'PowerPoint', or a .pptx filename.

Key capabilities:

- Create from scratch with PptxGenJS templates
- Edit existing presentations by unpacking XML and targeted edits
- Text extraction (markdown), visual thumbnails, overlap detection & auto-fix
- Image handling with aspect-ratio preservation; font detection
- Full QA loop: convert to images → inspect → fix → re-verify

5. skill-creator

Location: /root/.grok/skills/skill-creator/

Description: Guide for creating and updating skills that extend the agent's capabilities. Use when the user wants to create a new skill, update an existing skill, or asks about the skill format. Triggers include 'create a skill', 'make a skill for', 'new skill', 'update this skill', 'skill format'.

Key capabilities:

- SKILL.md frontmatter rules (name, description, allowed keys)
- Progressive disclosure model (metadata → body → resources)
- Resource guidelines: scripts/, references/, assets/
- Init & validate scripts; location rules (user skills persist in /home/workdir/.grok/skills/)
- Anti-patterns to avoid (duplicating model knowledge, nested references, etc.)

6. xlsx

Location: /root/.grok/skills/xlsx/

Description: Use any time a spreadsheet file is the primary input or output — open, read, edit, or fix .xlsx / .xlsm / .csv / .tsv files; create new spreadsheets; convert between tabular formats; clean messy data. Triggers on 'Excel', 'spreadsheet', 'xlsx', 'workbook', or references to spreadsheet files. Do **not** trigger when the primary deliverable is a Word doc, HTML report, standalone script, or Google Sheets API work.

Key capabilities:

- Create/edit with openpyxl (formulas, formatting) or pandas (analysis)
- Always prefer Excel formulas over hardcoded calculated values
- Formula recalculation via scripts/recalc.py (LibreOffice)
- Financial-model conventions: blue inputs, black formulas, yellow key assumptions
- Number formatting standards (currency, percentages, multiples, parentheses for negatives)

User Custom Skills

These skills live under `/home/workdir/.grok/skills/` and persist across sessions.

7. coding-agents-harness

Location: `/home/workdir/.grok/skills/coding-agents-harness/`

Description: Unified harness and launcher for multiple AI coding agent CLIs installed in the environment (Hermes Agent, Claude Code, Codex, OpenCode, OpenClaw, Muse Code, DeepSeek Harness). Use when the user wants to run, switch, compare, or manage coding agents, launch a specific CLI, list available agents, or perform coding tasks via any of these tools.

Installed agents:

Agent	Command	Notes
Hermes Agent	hermes	Nous Research self-improving agent
Claude Code	claude	Anthropic terminal coding agent
Codex	codex	OpenAI Codex CLI
OpenCode	opencode	OpenCode AI coding agent
OpenClaw	openclaw	Personal AI assistant / multi-provider
Muse Code	muse	Meta Muse Code (Muse Spark 1.2)
DeepSeek Harness	dsh	DeepSeek plugin-based agent harness

- Helper scripts: `list-agents.sh`, `launch-agent.sh`; detailed flags in `references/agent-commands.md`

8. live-preview

Location: `/home/workdir/.grok/skills/live-preview/`

Description: Provides live preview for HTML/CSS/JS or simple React projects by generating files and serving them locally with auto-reload. Use when the user wants to preview web code snippets, projects, or React components in the sandbox. Triggers: `live preview`, `preview html`, `react`, `serve`, `web demo`.

Key capabilities:

- Plain HTML/CSS/JS: write files → `python -m http.server` (or `npx serve`)
- React: Vite project (create + install + `npm run dev -- --host`)
- Helper: `scripts/start-preview.sh [dir] [port]`
- Common ports: 3000, 5173, 8000; clean dedicated project dirs recommended

9. nvidia-llm

Location: `/home/workdir/.grok/skills/nvidia-llm/`

Description: Call NVIDIA Integrate API (OpenAI-compatible chat completions) with models `stepfun-ai/step-3.7-flash` and `minimaxai/minimax-m3`. Use when the user asks to query an NVIDIA model, try `stepfun` or `minimax`, use an alternative LLM provider, or run chat completions against `integrate.api.nvidia.com`. Also triggers on slash-style commands such as `/nvidia @minimax`, `/nvidia @step`, `/nvidia minimax`, `/nvidia stepfun`. In those cases act as a thin proxy and return the model reply with minimal mediation.

Key capabilities:

- Direct proxy mode for `/nvidia` commands (output only the model tokens)
- Models: `minimaxai/minimax-m3` (default), `stepfun-ai/step-3.7-flash`
- Helper script: `scripts/chat.py --model MODEL_ID --message "PROMPT"`
- Endpoint: `https://integrate.api.nvidia.com/v1/chat/completions`
- Options: system prompt, max-tokens, temperature, top-p, raw JSON

End of catalog — 6 bundled + 3 user custom skills
For full procedural details, open each skill's SKILL.md file.